Министерство науки и высшего образования Российской Федерации
Новосибирский государственный технический университет «НЭТИ»

Кафедра теоретической и прикладной информатики

Лабораторная работа № 3 по дисциплине «Низкоуровневое программирование» на тему «Интерфейс с языком C++»

Факультет прикладной математики и информатики
Группа: ПМИ-43
Бригада: 5
Студенты: Павлова С. К., Окунева П. И., Мохначев С. С.
Преподаватель: Сивак М. А.

Новосибирск, 2026

Цель работы

Изучить и приобрести практические навыки использования процедур на языке Ассемблера и их интерфейса с языком C++.

Формулировка задачи

1. Написать процедуру на языке Ассемблера, реализующую функцию заданного варианта с использованием цепочечных команд. Написать вызывающую функцию на языке C++, осуществляющую ввод исходных данных и вывод результатов.

2. Отладить программу, убедиться в правильности её работы на тестовых примерах.

Данные варианта 5:

«Написать программу, выделяющую из исходной строки подстроку заданной длины с указанного номера позиции и приводящую символы этой подстроки к верхнему регистру».

Используемое соглашение о вызовах: cdecl

Передача данных в процедуру на языке Ассемблера осуществляется через аргументы.

Решение задачи

Соглашение cdecl означает, что при вызове процедуры, написанной на языке Ассемблера, аргументы передаются ей по значению. Аргументы помещаются в стек в порядке, обратном тому, в котором они указаны при вызове процедуры. По окончании выполнения подпрограммы программа очищает стек от аргументов.

Опишем алгоритм решения задачи варианта 5 (то, что делает процедура на языке Ассемблера).

Исходные данные:

1) исходная строка SOURCE, представляющая собой массив байтов;

2) позиция POS, начиная с которой выделяется подстрока - целое число;

3) длина LEN нужной подстроки - целое число;

4) указатель на символьный массив DEST для записи по этому адресу результата.

Результат работы программы:

1) подстрока исходной строки, приведенная к верхнему регистру.

Алгоритм

1) Передать данные в подпрограмму через аргументы

2) Сбросить значение флага направления DF для прохода от начала цепочки до ее конца в сторону увеличения адресов

3) Загрузить адрес начала подстроки в регистр ESI (исходная строка - источник данных)

4) Загрузить адрес буфера результата в регистр EDI (результат - приемник данных)

5) Загрузить длину LEN в регистр ECX, содержимое которого выступает в роли счетчика

6) Пока ECX != 0:

- загрузить байт из подстроки в регистр AL, сдвинуть адрес на 1 вправо;
- если значение в AL в диапазоне ‘a’, …, ‘z’, то привести значение в AL к верхнему регистру;

записать AL по адресу из EDI, сдвинуть EDI на 1.

7) Результат расположен по адресу [EDI].

Модуль программы на C++ (файл lab_3_np.cpp):

```
#include <iostream>#include <cstdio>#include <string>
extern "C" void SUBSTR(const char* SOURCE, int POS, int LEN, char* DEST);
int main() {
	std::string SOURCE;
	int POS;
	int LEN;
	char DEST[200] = {0};

	std::cout << "Enter string" << std::endl;
	std::getline(std::cin, SOURCE);

	std::cout << "Enter position in string (starting from 0)" <<
std::endl;
	std::cin >> POS;

	std::cout << "Enter length of substring" << std::endl;
	std::cin >> LEN;

	if (POS < 0 || LEN < 0 || POS + LEN >
static_cast<int>(SOURCE.length())) {
		std::cout << "Incorrect data" << std::endl;
		return 1;
	}

	SUBSTR(SOURCE.c_str(), POS, LEN, DEST);

	std::cout << "Result:" << std::endl;
	std::cout << DEST << std::endl;

	return 0;
}
```

Компилятор Visual C++ после вызова процедуры SUBSTR генерирует ассемблерную команду вида

CALL _SUBSTR

ADD ESP, 16

и во второй строке, после выполнения процедуры SUBSTR, происходит очистка стека от переданных параметров. Их всего 4, каждый занимает по 4 байта (целые числа передаются копией, а массивы символов передаются по адресу, который указывает на первый элемент и занимает тоже 4 байта), итого 16 байтов нужно очистить.

Модуль подпрограммы на языке Ассемблера (main.asm):

```
.386
.MODEL FLAT

.CODE
_SUBSTR PROC                    ; имя с нижним подчеркиванием соответствует
                                ; глобальному идентификатору,
                                ; определенному в программе
     PUSH EBP                   ; сохраняем значение EBP, т.к. подпрограмма
                                ; не должна его изменять.
                                ; используем его как указатель базы стека
     MOV EBP, ESP               ; в ESP лежит адрес возврата в программу
     MOV ESI, [EBP + 8]         ; в стеке по этому адресу лежит адрес
                                ; первого символа строки SOURCE
     MOV EAX, [EBP + 12]        ; POS
     ADD ESI, EAX               ; теперь ESI указывает на первый символ
                                ; подстроки
     MOV EDI, [EBP + 20]        ; тут адрес буфера DEST
     MOV ECX, [EBP + 16]        ; тут длина LEN
     CLD

L:
     LODSB                      ; загрузит текущий элемент цепочки по
                                ; адресу из ESI в регистр AL,
                                ; т.к. работаем с байтами
     CMP AL, 'a'                ; проверка, что символ в диапазоне от 'a'
                                ; до 'z'
     JB STORE
     CMP AL, 'z'
     JA STORE                   ; если в диапазоне, то переходим на метку
                                ; STORE
     AND AL, 11011111B          ; если нет, то с помощью логического
                                ; побитового умножения и маски
                                ; 11011111 обнуляем пятый разряд символа из
                                ; AL
STORE:
     STOSB                      ; сохранит элемент цепочки из AL по адресу
                                ; из EDI, а там у нас буфер
     LOOP L
     MOV BYTE PTR [EDI], 0
     POP EBP                    ; восстанавливаем значение EBP
     RET                        ; нет аргументов у RET, так как стек, по
                                ; соглашению cdecl, очищает программа, а не
                                ; подпрограмма
_SUBSTR ENDP
END
```

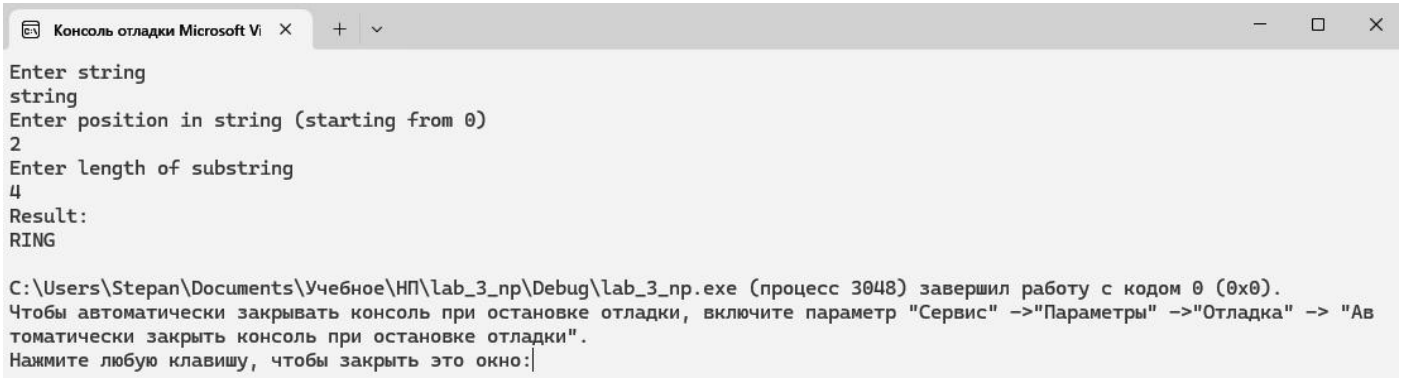
Тестирование программы

1) Тест на работоспособность

SOURCE: string

POS: 2

LEN: 4



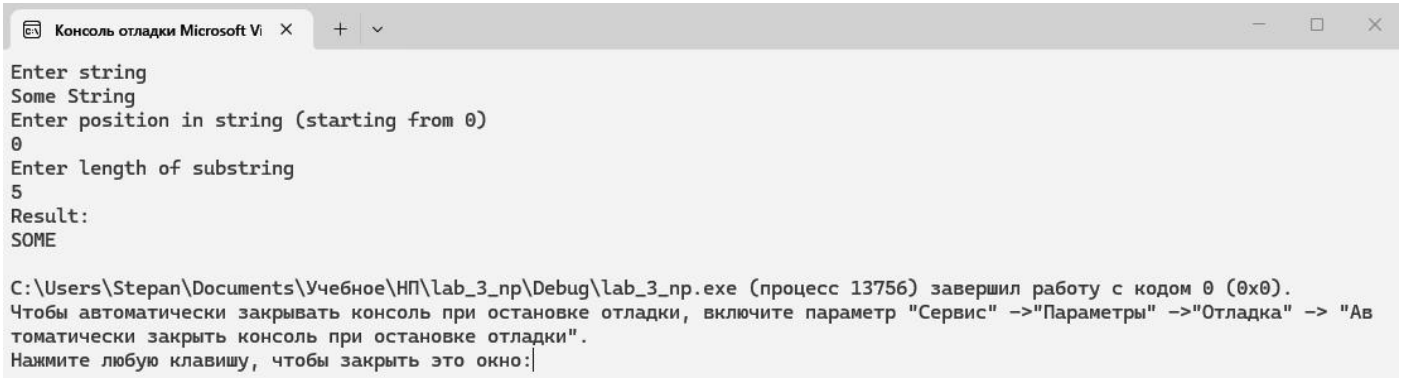
результат: RING

2) Уже есть заглавные буквы и есть пробел

SOURCE: Some String

POS: 0

LEN: 5



результат: SOME

С помощью команды AND (логическое И, или логическое произведение) и маски 11011111B в качестве операнда-источника производится сброс 5-го разряда двоичного числа (байта) и получается верхний регистр. Это работает, потому что 5-ый разряд у строчных букв равен 1, а у заглавных - 0, и это справедливо для всех латинских букв. Поскольку проверка на принадлежность символа промежутку ‘a’, …, ‘z’выполняется перед выполнением команды AND, эта команда никак не подействует на другие

символы (прописные буквы и небуквенные символы). Это продемонстрировано подробнее в следующих тестах

3) Смешанный регистр

SOURCE: StRiNg

POS: 0

LEN: 6

```
Enter string
StRiNg
Enter position in string (starting from 0)
0
Enter length of substring
6
Result:
STRING

C:\Users\Stepan\Documents\Учебное\НП\lab_3_np\Debug\lab_3_np.exe (процесс 10296) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Ав
томатически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

результат: STRING

4) Есть прочие символы

SOURCE: str14!#

POS: 0

LEN: 7

```
Enter string
str14!#
Enter position in string (starting from 0)
0
Enter length of substring
7
Result:
STR14!#

C:\Users\Stepan\Documents\Учебное\НП\lab_3_np\Debug\lab_3_np.exe (процесс 11696) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Ав
томатически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

результат: STR14!#

Вывод

Изучены способы вызова процедур (подпрограмм) на языке Ассемблера и возврата из процедур, а также правила взаимодействия функций C++ с процедурами языка Ассемблера. В программе использовано соглашение о вызовах cdecl, согласно которому передача параметров происходила через стек в обратном порядке и очисткой стека занималась программа.

Реализована подпрограмма на языке Ассемблера, решающая задачу варианта. Написана вызывающая программа на языке C++, осуществляющая ввод данных и их проверку и вывод результата работы подпрограммы. Проведены тесты.